

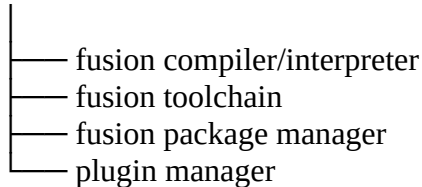
Hello and thank you for using fusion. It is my privilege to introduce you to my programming language.

Fusion acts as an interoperability layer between software ecosystems. Fusion is a programming language designed around ecosystem interoperability.

You can consider fusion a **unified software development platform with a language attached.**

When you install fusion (vanilla or basic), you get something like this:

Fusion installation



There are 2 types of plugins(THAT STILL NEED TO BE BUILT), the first type are **ergonomic in nature**, they make development/programming more efficient, pleasant and easier to maintain. These plugins include SQLite, UI framework, cloud builds, REPL+jupyter, JIT+AOT, actors system(from erlang/elixir) and more.

The second type **increase the access and expands the reach of fusion**. It allows fusion to treat foreign programming languages as resources to be exploited and expanded. By integrating or embedding foreign source code files, fusion can compile/build files written in another programming language with minimal overhead, no rewrite needed, let alone installing and maintaining a foreign programming language.

Ask yourself, if you can build it in Fusion, if the answer is no, which ecosystem is the best implementation for this specific capability?

Your knowledge should be transferable to fusion, regardless which programming language you use.

I am pushing Fusion toward being an **orchestration/interoperability language**, rather than merely another general-purpose language.

The language is the interface.

The compiler is the execution mechanism.

Foundry is the ecosystem coordinator(package manager).

Interoperability is the reason the whole thing exists.

Fusion may suggest to you that a function written in fusion would be faster or take less memory in another programming language like C and suggest to move it to a C file.

The Fusion Programming Language

Language Guide and Reference

-----qualities and characteristics-----

Fusion has:

- static typing with type inference,
- variables are mutable by default,
- with strong typing.

Fusion is meant to be very modern and beginner friendly.

You can make comments with `<///>` or `<#>`, for multi-line comments use `</**/>`.

You can use indentation or braces, whatever you like.

To use indentation write `main:`

To use braces write `main{`

Remember there is no mixing within the same file!

The GC is responsible for ordinary memory, while »defer« is responsible for deterministic cleanup of resources that need it. I'm deliberately separating **memory lifetime** from **resource lifetime**.

That gives Fusion something closer to:

GC for memory + RAII-like deterministic cleanup for resources

Fusion will have a package manager similar to Cargo from rust, fusion's package manager is called **foundry**.

There is no need for dictionaries, all you need is a growable array and a hash map for everything. Fusion also has `match`, `enums`, `structs`, `impl`, `traits`, `iterators` borrowed from rust.

Fusion has many operations for growable arrays, hash maps and generic rust-style iterators.

- growable arrays(+operations): `create`, `add`, `remove_last`, `access`, `modify`, `get_length`, `iterate`, `clear`, `insert_at_index`, `remove_at_index`.
- hash map(+operations): `create`, `insert`, `get`, `remove`, `contains_key`, `iterate`, `clear`, `iterate`, `keys`, `values`.
- iterators(+operations): `map`, `filter`, `enumerate`, `collect`, `find`, `any`, `all`, `fold`.
- string operations(+operations): `create`, `concatenate`, `substring`, `find`, `replace`, `split`, `trim`, `to_uppercase`, `to_lowercase`, `starts_with`, `ends_with`, `contains`, `length`, `iterate`.

It also has: async/await, Filesystem, Command-line arguments, Process/system functionality, Time and dates, Randomness, networking, serialisation, formating, encoding, logging.